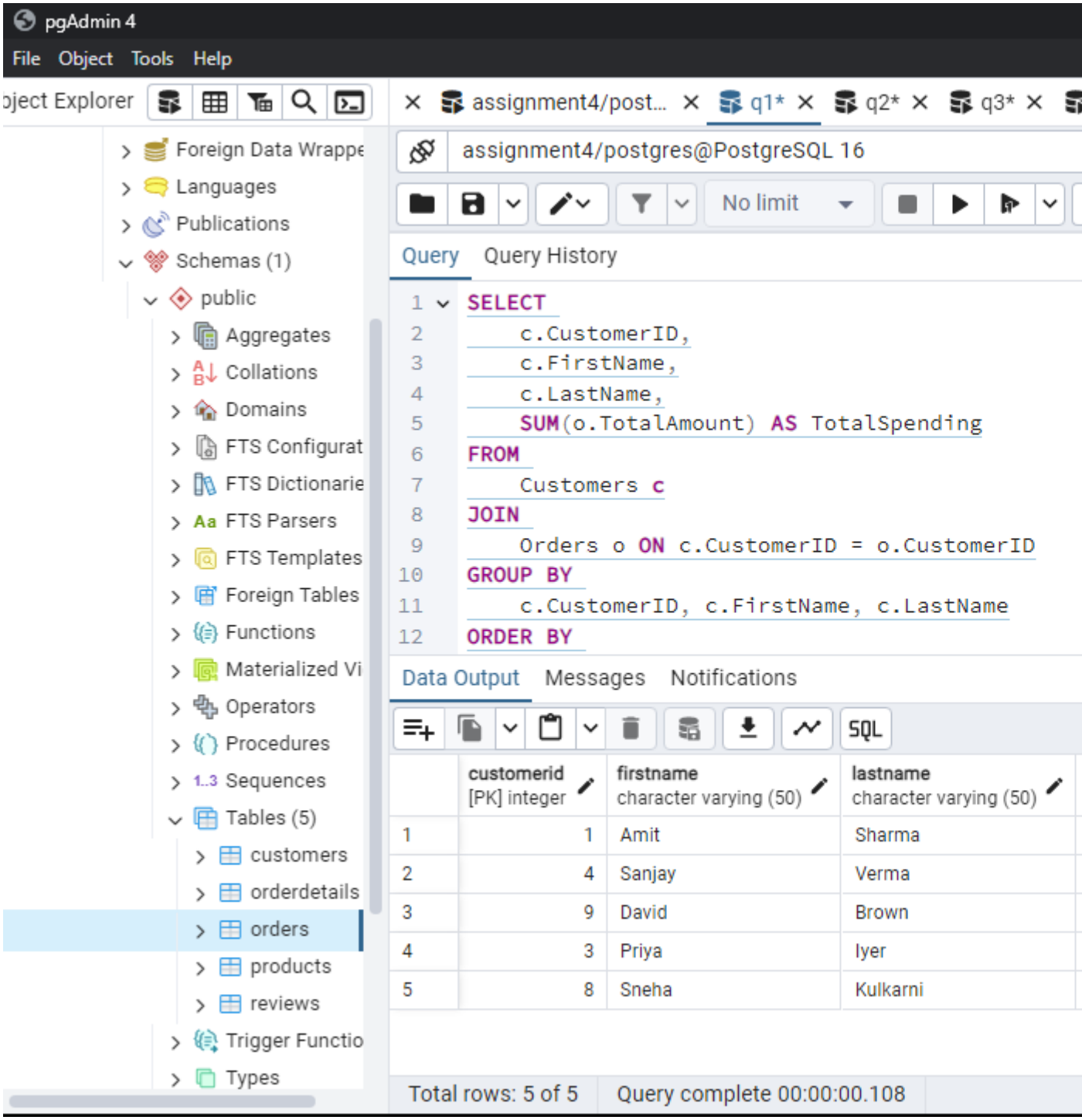
Questions:

1. Top 5 Customers by Total Spending:

- Use Select, Join, and Aggregation to identify the top 5 customers who have spent the most on the platform. The total spending should be calculated based on the sum of all orders per customer.

SQL Code:

```
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName,
    SUM(o.TotalAmount) AS TotalSpending
FROM
    Customers c
JOIN
    Orders o ON c.CustomerID = o.CustomerID
GROUP BY
    c.CustomerID, c.FirstName, c.LastName
ORDER BY
    TotalSpending DESC
LIMIT 5;
```



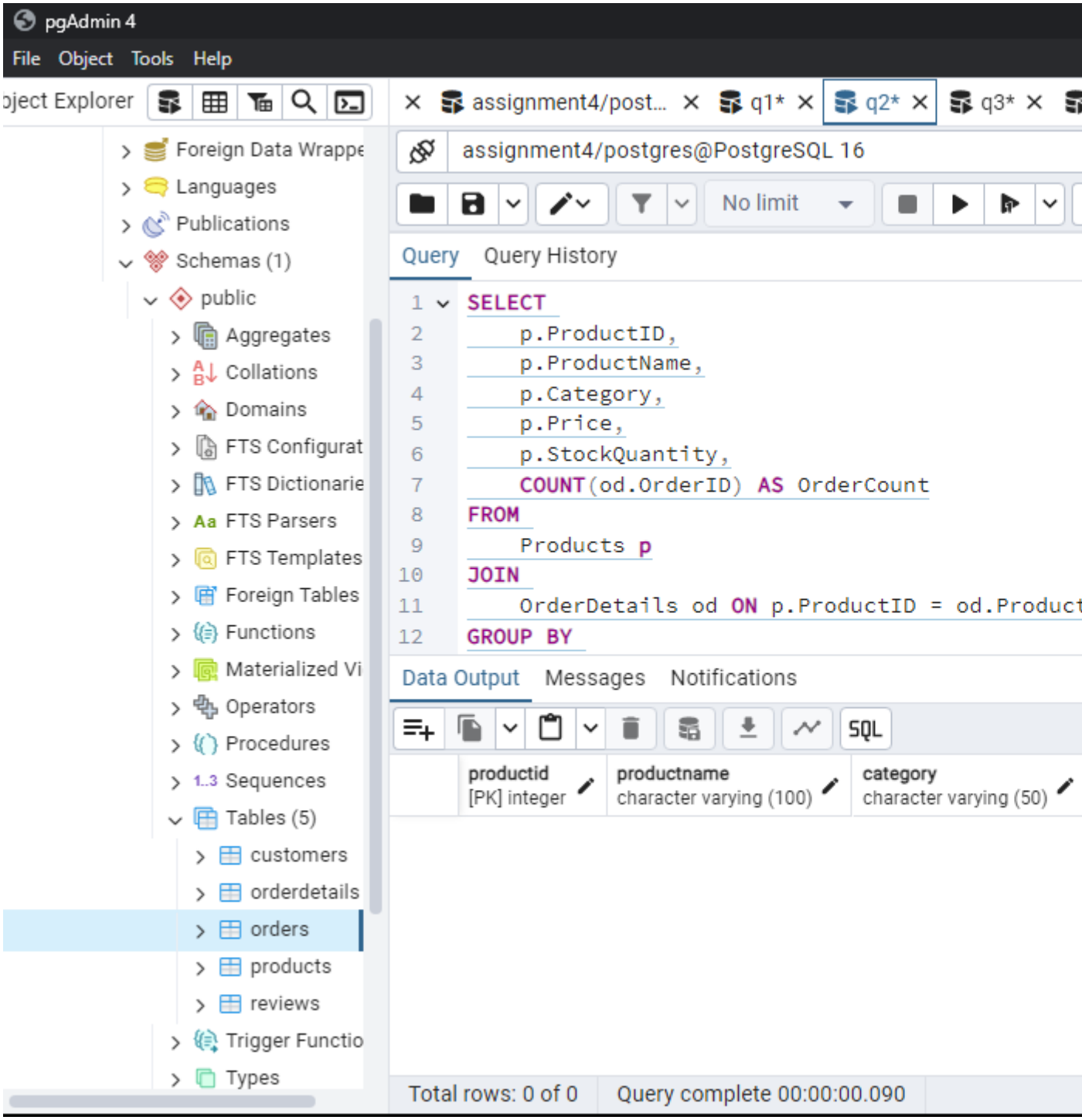
2. Products with Low Stock and High Demand:

- Write a query using Select, Join, Aggregation, and Having to find products that have less than 10 units in stock but have been ordered more than 2 times. Utilize Join to combine the Products and OrderDetails tables.

SQL Code:

SELECT

```
    p.ProductID,
    p.ProductName,
    p.Category,
    p.Price,
    p.StockQuantity,
    COUNT(od.OrderID) AS OrderCount
FROM
    Products p
JOIN
    OrderDetails od ON p.ProductID = od.ProductID
GROUP BY
    p.ProductID, p.ProductName, p.Category, p.Price, p.StockQuantity
HAVING
    p.StockQuantity < 10
    AND COUNT(od.OrderID) > 2;
```



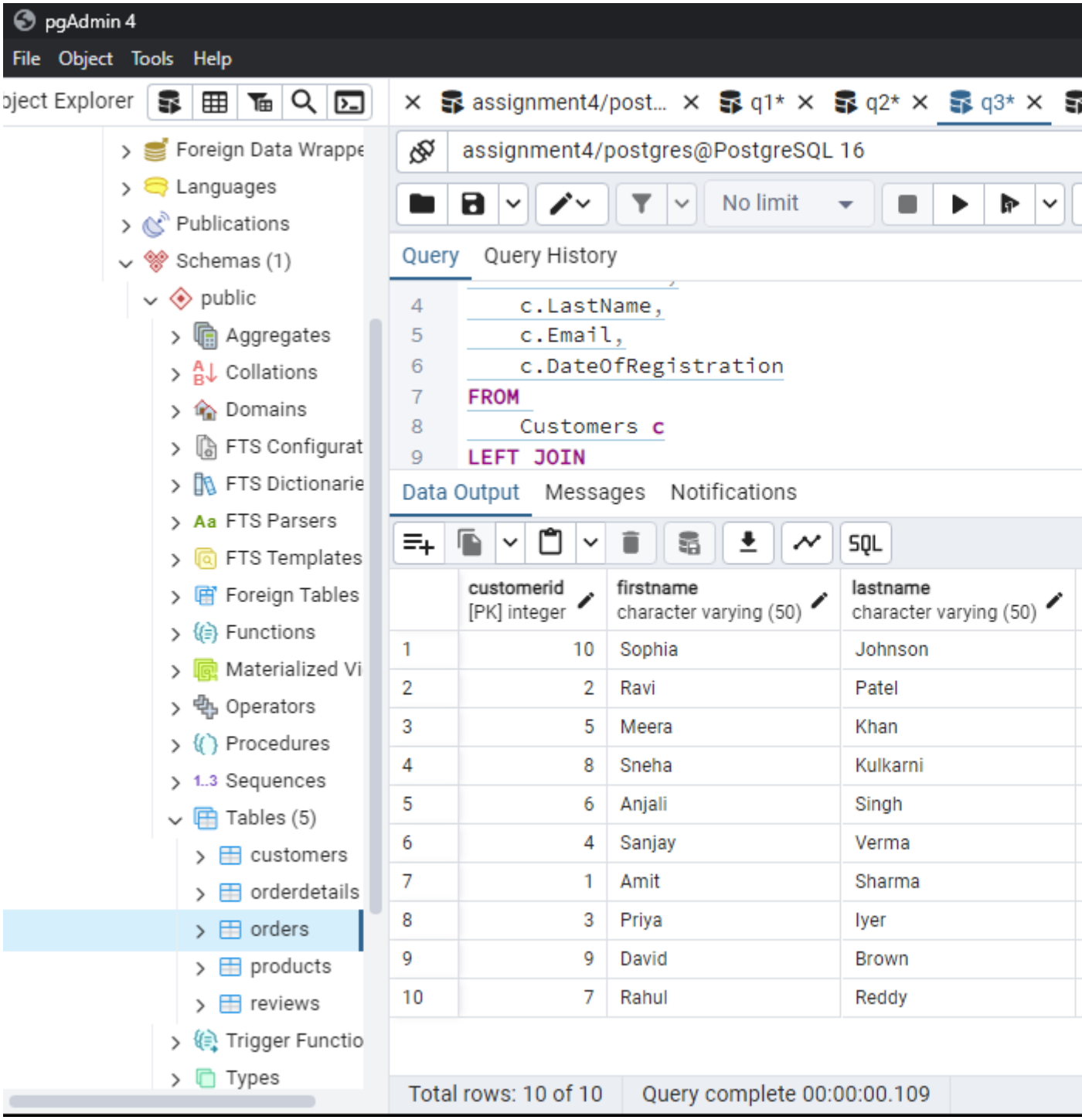
3. Customers Who Haven’t Ordered in the Last 6 Months:

- Use Minus (EXCEPT) or LEFT JOIN along with Select to find customers who registered but haven't placed orders in the last 6 months.

SQL Code:

```
SELECT
    c.CustomerID,
```

```
    c.FirstName,
    c.LastName,
    c.Email,
    c.DateOfRegistration
FROM
    Customers c
LEFT JOIN
    Orders o ON c.CustomerID = o.CustomerID
    AND o.OrderDate >= CURRENT_DATE - INTERVAL '6 months'
WHERE
    o.OrderID IS NULL;
```



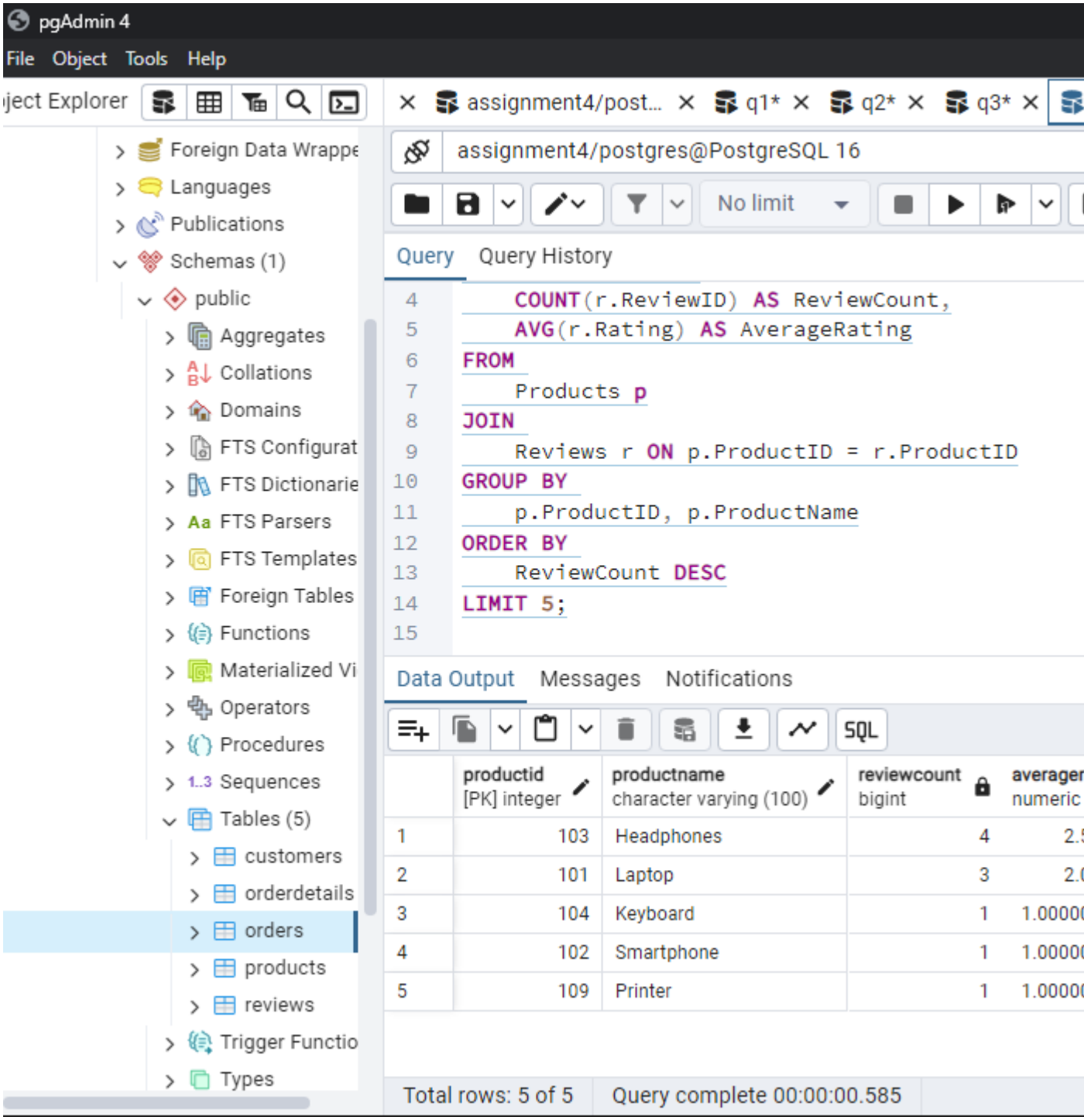
4. Most Reviewed Products with Average Rating:

- Retrieve the top 5 most reviewed products along with their average rating. Use Select to project columns, Join the Products and Reviews tables, apply Aggregation (COUNT, AVG), and use ORDER BY to rank the products by the number of reviews.

SQL Code:

SELECT

```
    p.ProductID,
    p.ProductName,
    COUNT(r.ReviewID) AS ReviewCount,
    AVG(r.Rating) AS AverageRating
FROM
    Products p
JOIN
    Reviews r ON p.ProductID = r.ProductID
GROUP BY
    p.ProductID, p.ProductName
ORDER BY
    ReviewCount DESC
LIMIT 5;
```



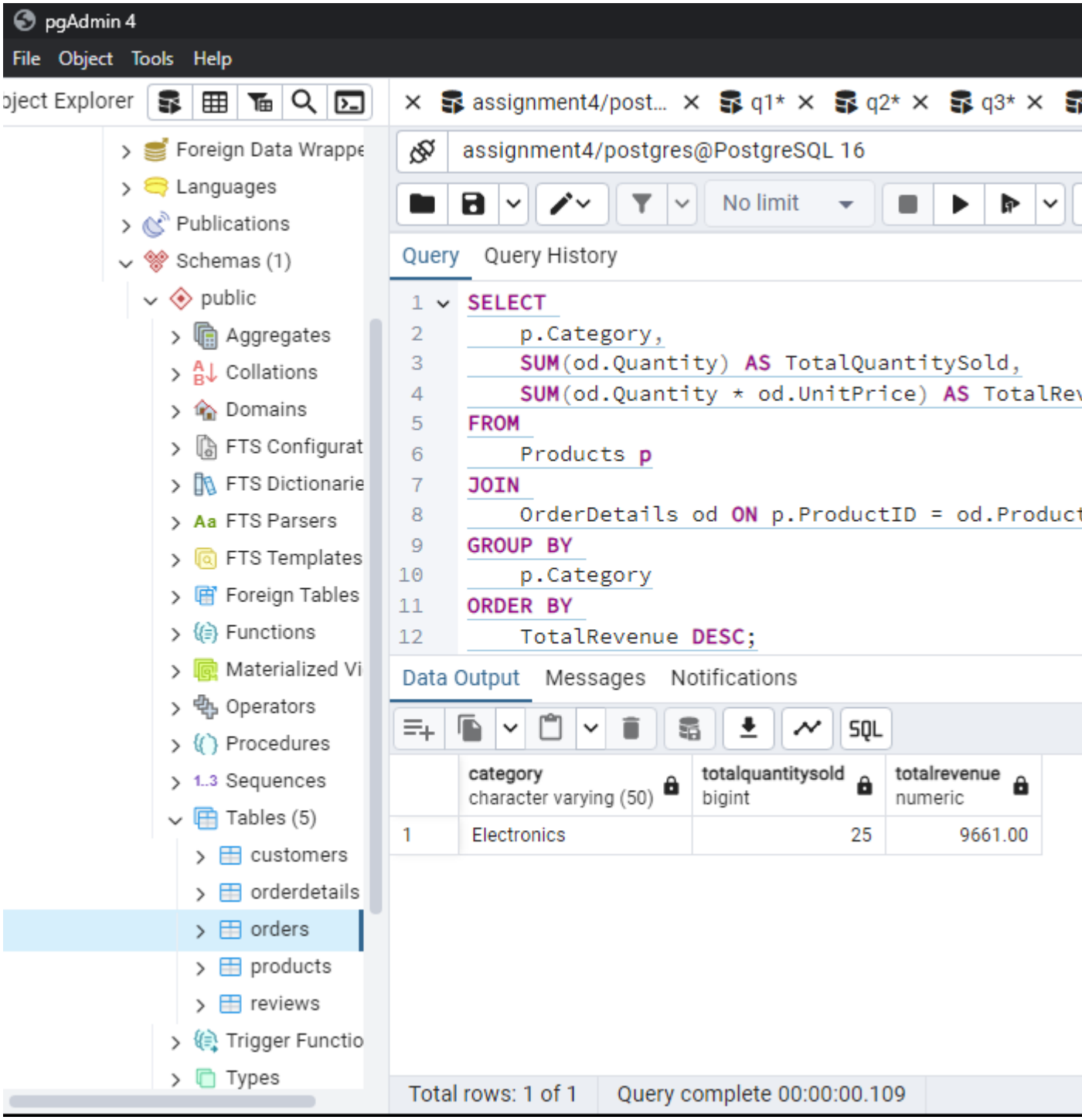
5. Total Sales by Product Category:

- Write a query using Select, Join, and Aggregation to calculate the total quantity sold and total revenue generated for each product category.

SQL SELECT

p.Category,

```
    SUM(od.Quantity) AS TotalQuantitySold,
    SUM(od.Quantity * od.UnitPrice) AS TotalRevenue
FROM
    Products p
JOIN
    OrderDetails od ON p.ProductID = od.ProductID
GROUP BY
    p.Category
ORDER BY
    TotalRevenue DESC;
```



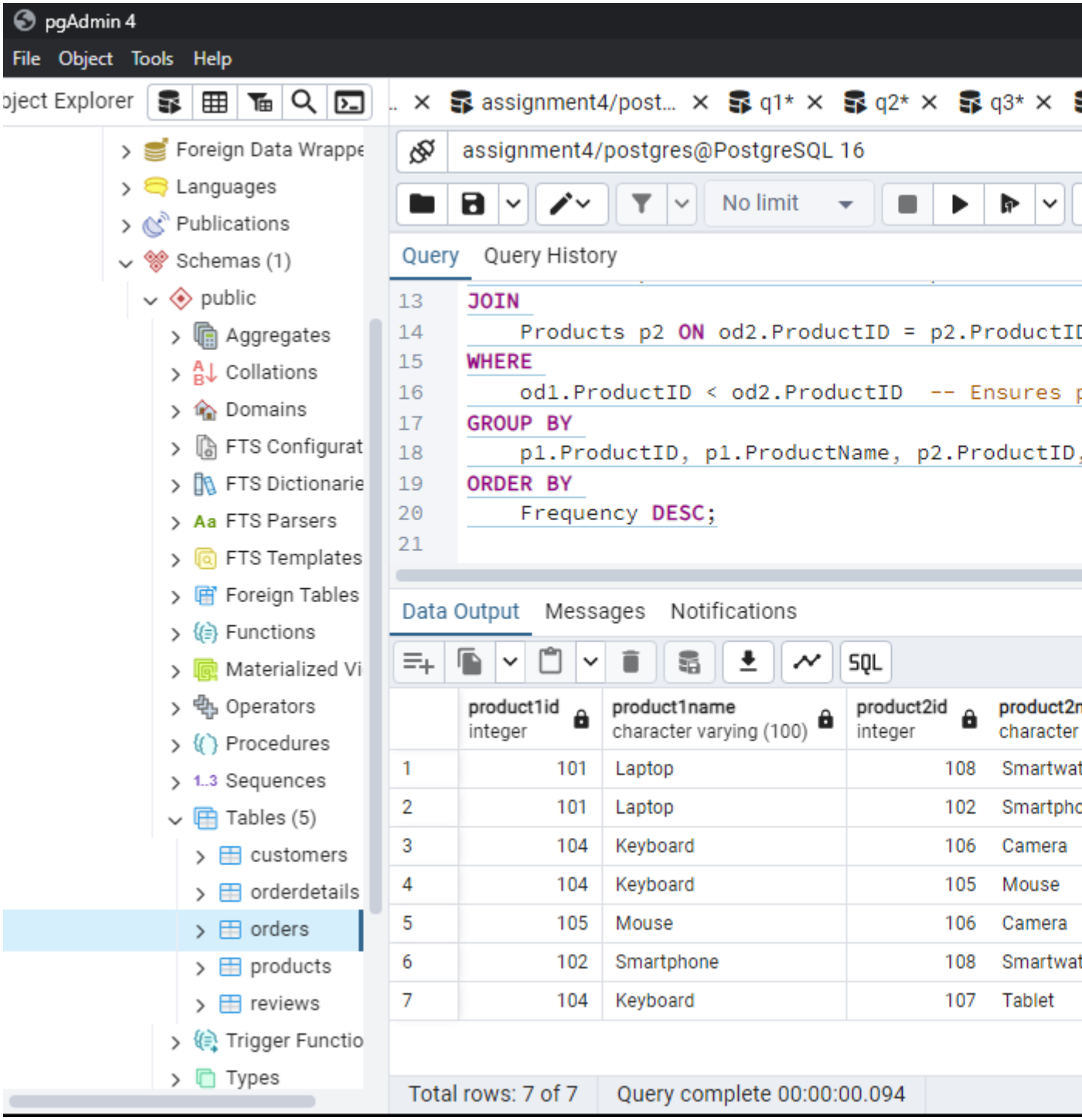
6. Frequently Purchased Together Products:

- Use Self-Join and Cartesian Cross Product to find pairs of products that are frequently purchased together in the same order. Ensure that product pairs are distinct and ordered by frequency.

SQL Code:

SELECT

```
    p1.ProductID AS Product1ID,
    p1.ProductName AS Product1Name,
    p2.ProductID AS Product2ID,
    p2.ProductName AS Product2Name,
    COUNT(*) AS Frequency
FROM
    OrderDetails od1
JOIN
    OrderDetails od2 ON od1.OrderID = od2.OrderID
JOIN
    Products p1 ON od1.ProductID = p1.ProductID
JOIN
    Products p2 ON od2.ProductID = p2.ProductID
WHERE
    od1.ProductID < od2.ProductID  -- Ensures pairs are distinct and avoids double counting
GROUP BY
    p1.ProductID, p1.ProductName, p2.ProductID, p2.ProductName
ORDER BY
    Frequency DESC;
```



7. Customers Who Left Only Negative Reviews:

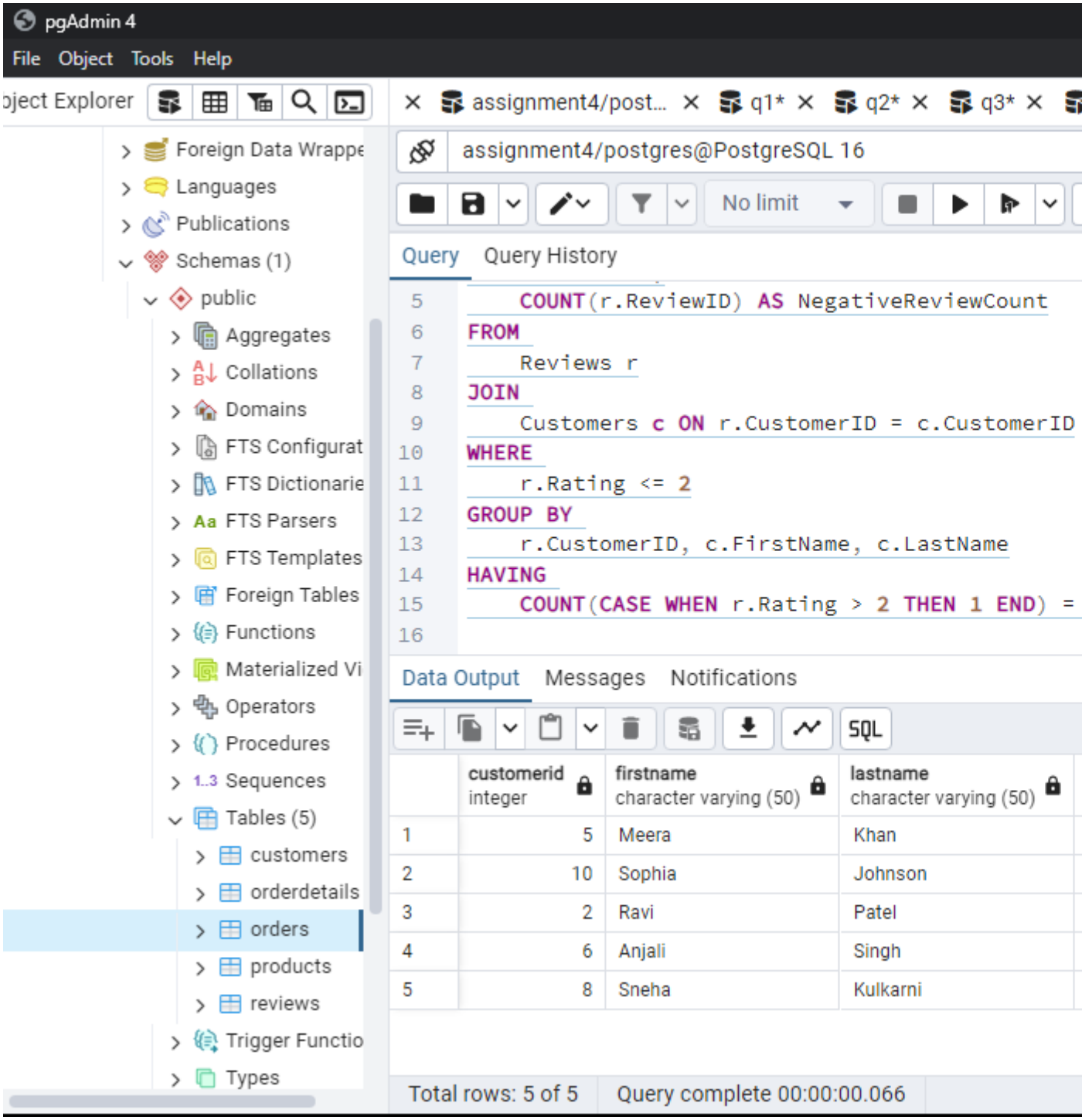
- Use Select, Join, and Grouping to identify customers who have left only negative reviews

(ratings of 1 or 2) and no positive reviews.

SQL Code:

```
SELECT
    r.CustomerID,
```

```
    c.FirstName,
    c.LastName,
    COUNT(r.ReviewID) AS NegativeReviewCount
FROM
    Reviews r
JOIN
    Customers c ON r.CustomerID = c.CustomerID
WHERE
    r.Rating <= 2
GROUP BY
    r.CustomerID, c.FirstName, c.LastName
HAVING
    COUNT(CASE WHEN r.Rating > 2 THEN 1 END) = 0;
```



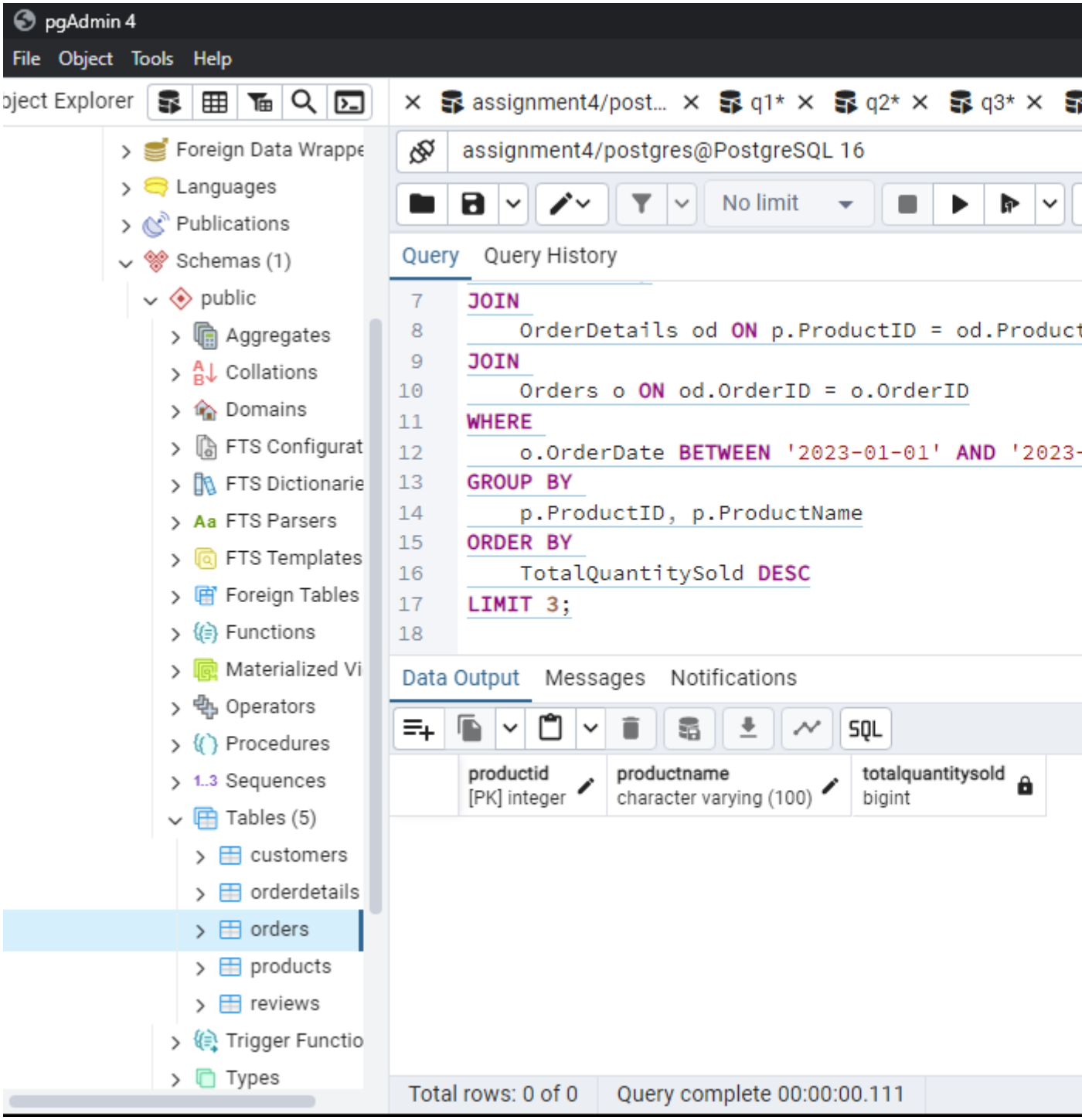
8. Best-Selling Products in a Time Range:

- Find the top 3 best-selling products by quantity sold between January 1, 2023, and June 30, 2023. Use Select to project fields, Join relevant tables, and apply Aggregation (SUM). Use WHERE to filter by date and ORDER BY to rank by quantity.

SQL Code:

SELECT

```
    p.ProductID,
    p.ProductName,
    SUM(od.Quantity) AS TotalQuantitySold
FROM
    Products p
JOIN
    OrderDetails od ON p.ProductID = od.ProductID
JOIN
    Orders o ON od.OrderID = o.OrderID
WHERE
    o.OrderDate BETWEEN '2023-01-01' AND '2023-06-30'
GROUP BY
    p.ProductID, p.ProductName
ORDER BY
    TotalQuantitySold DESC
LIMIT 3;
```



9. Intersection of Customers with Orders and Reviews:

- Use Intersection (INTERSECT) to find customers who have both placed orders and written reviews.

-- Subquery to find customers who have placed orders

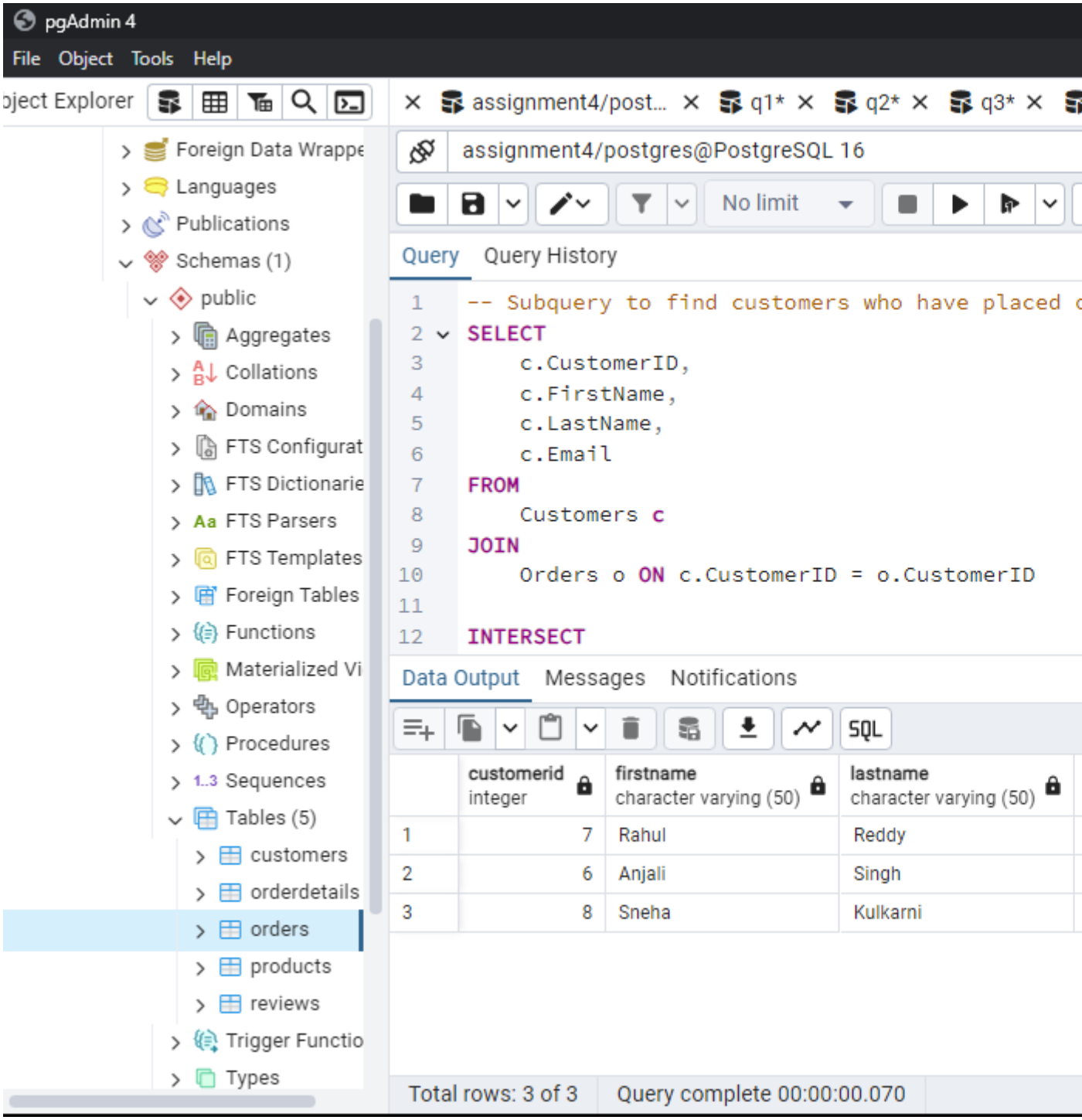
SQL Code:

```
SELECT
    c.CustomerID,
```

```
    c.FirstName,
    c.LastName,
    c.Email
FROM
    Customers c
JOIN
    Orders o ON c.CustomerID = o.CustomerID

INTERSECT

-- Subquery to find customers who have written reviews
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName,
    c.Email
FROM
    Customers c
JOIN
    Reviews r ON c.CustomerID = r.CustomerID;
```



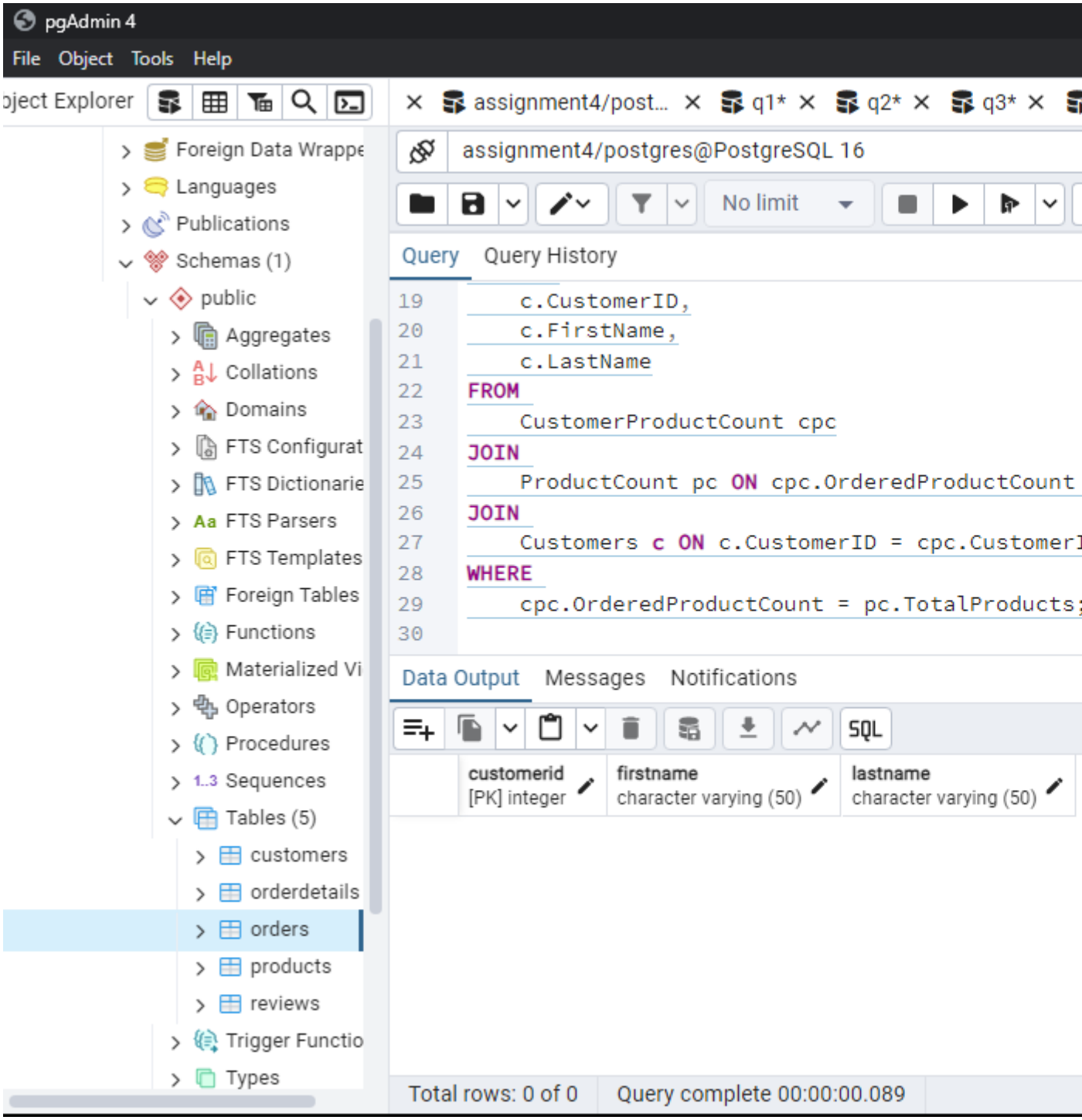
10. Division Operation: Customers Who Ordered All Products:

- Implement the Division operation to find customers whose order history covers all products.

SQL Code:

```
WITH ProductCount AS (
    SELECT
        COUNT(*) AS TotalProducts
```

```
    FROM
        Products
),
CustomerProductCount AS (
    SELECT
        o.CustomerID,
        COUNT(DISTINCT od.ProductID) AS OrderedProductCount
    FROM
        Orders o
    JOIN
        OrderDetails od ON o.OrderID = od.OrderID
    GROUP BY
        o.CustomerID
)
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName
FROM
    CustomerProductCount cpc
JOIN
    ProductCount pc ON cpc.OrderedProductCount = pc.TotalProducts
JOIN
    Customers c ON c.CustomerID = cpc.CustomerID
WHERE
    cpc.OrderedProductCount = pc.TotalProducts;
```


pgAdmin 4
File Object Tools Help
Object Explorer
assignment4/post...
q1*
q2*
q3*
Foreign Data Wrappe
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configurat
FTS Dictionarie
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Vi
Operators
Procedures
Sequences
Tables (5)
customers
orderdetails
orders
products
reviews
Trigger Functio
Types
assignment4/postgres@PostgreSQL 16
No limit
Query
Query History
19 c.CustomerID,
20 c.FirstName,
21 c.LastName
22 FROM
23 CustomerProductCount cpc
24 JOIN
25 ProductCount pc ON cpc.OrderedProductCount
26 JOIN
27 Customers c ON c.CustomerID = cpc.CustomerI
28 WHERE
29 cpc.OrderedProductCount = pc.TotalProducts;
30
Data Output
Messages
Notifications
SQL
customerid
[PK] integer
firstname
character varying (50)
lastname
character varying (50)
Total rows: 0 of 0
Query complete 00:00:00.089